

PORTFOLIO · IMDAD BOURAIMA · 2026

Projet n°07 — WEB · MVC · RÉSERVATION

Neige & Soleil

Client léger MVC — Plateforme de location de chalets en montagne

PHP · Architecture MVC · MySQL · JavaScript · HTML/CSS · PDO

1. ÉTAT DES LIEUX — PRÉSENTATION DE NEIGE & SOLEIL

Neige & Soleil est une agence de location de logements en montagne proposant des studios, appartements et chalets dans les Alpes. Elle gère un parc de biens immobiliers appartenant à des propriétaires privés, et les met en location auprès de vacanciers (clients) via une plateforme web.

Le projet "Client Léger" est une application web développée en architecture MVC (Modèle-Vue-Contrôleur) en PHP. Elle centralise la gestion des réservations en ligne, la consultation du catalogue de logements, la facturation automatique et le suivi du profil client. L'ensemble du projet repose sur un seul point d'entrée : `index.php`, qui orchestre toutes les actions via un switch sur le paramètre GET page.

CONTEXTE ET UTILISATEURS

- Clients (vacanciers) : consultent le catalogue, réservent des logements, gèrent leur panier et consultent leurs factures
- Propriétaires : gèrent leur portefeuille de biens depuis leur espace dédié (ajout, modification, suppression d'appartements)
- Administrateurs / Personnel de l'agence : gestion interne des réservations et des contrats

BESOIN IDENTIFIÉ — LA PROBLÉMATIQUE

L'agence Neige & Soleil ne disposait pas d'outil numérique permettant aux clients de consulter les logements disponibles et de réserver en ligne. Les propriétaires ne pouvaient pas gérer leur portefeuille à distance. L'application web devait répondre à ces deux besoins tout en produisant des factures automatiques lors de la validation d'une réservation.

MISSION RÉALISÉE

Développement d'une application web complète en architecture MVC PHP avec : catalogue de logements interactif, système de panier de réservation avec contrôle des chevauchements de dates, validation et facturation automatique, espace propriétaire avec CRUD complet, et gestion des utilisateurs via un système de triggers SQL pour l'héritage de tables.

2. OUTILS ET TECHNOLOGIES UTILISÉS

PHP — ARCHITECTURE MVC ET LOGIQUE SERVEUR

PHP est le langage serveur central de l'application. L'architecture MVC est implémentée avec trois composants distincts : le Modèle (modele.php) gère tous les accès à la base de données via PDO, le Contrôleur (controleur.class.php) fait le lien entre le modèle et les vues, et les Vues (dossier Vue/) affichent les données en HTML. Le fichier index.php est le point d'entrée unique (Front Controller) qui route les requêtes via un switch.

```
// index.php - Point d'entrée unique (Front Controller)
session_start();
require_once('Contrôleur/controleur.class.php');
$unContrôleur = new Contrôleur();

$page = isset($_GET['page']) ? $_GET['page'] : 'accueil';

switch($page) {
    case 'accueil': $appartementsVedettes = $unContrôleur->getAppartementsVedettes(); break;
    case 'appartements': $appartements = $unContrôleur->afficherCatalogue(); break;
    case 'panier': $lesReservations = $unContrôleur->getPanierByClient($_SESSION['id_user']); break;
    // ... autres cas
}include('Vue/vue_' . $page . '.php');
```

MYSQL — BASE DE DONNÉES AVEC HÉRITAGE PAR TRIGGERS

La base de données neige_soleil utilise une architecture d'héritage de tables gérée par des triggers SQL. La table User est la table mère centrale regroupant tous les utilisateurs (clients, propriétaires). L'insertion dans CLIENT ou PROPRIETAIRE déclenche automatiquement un trigger BEFORE INSERT qui insère les données dans User avec le rôle approprié. Cette architecture garantit l'unicité des comptes et simplifie l'authentification.

```
-- Trigger d'héritage : INSERT CLIENT → INSERT User automatique
CREATE TRIGGER insert_client BEFORE INSERT ON CLIENT
FOR EACH ROW BEGIN
    SET NEW.id_client = IFNULL((SELECT MAX(id_perso) FROM User), 0) + 1;
    INSERT INTO User (id_perso, nom, prenom, email, tel, role)
    VALUES (NEW.id_client, NEW.nom, NEW.prenom, NEW.email, NEW.tel, 'client');END;
```

PDO — ACCÈS SÉCURISÉ À LA BASE DE DONNÉES

Toutes les interactions avec la base de données passent par PDO (PHP Data Objects) avec des requêtes préparées (prepare/execute). Cette approche protège contre les injections SQL. La connexion est initialisée dans le constructeur de la classe Modèle et réutilisée pour toutes les requêtes via l'attribut privé \$pdo.

HTML5 / CSS3 / JAVASCRIPT — INTERFACE RESPONSIVE

Le front-end utilise HTML5 sémantique, CSS3 avec variables CSS, Flexbox et media queries pour un affichage responsive. Chaque page dispose de sa propre feuille de style dans le dossier Style/ (style_accueil.css, style_appartement.css, etc.). Font Awesome 6.0 est intégré via CDN pour les icônes. La navigation est assurée par un header commun (vue_header.php) présent sur toutes les pages sauf connexion et inscription.

3. ARCHITECTURE TECHNIQUE DU PROJET

Langage back-end	PHP — architecture MVC avec Front Controller (index.php)
Langage front-end	HTML5, CSS3 (Flexbox, variables CSS), JavaScript ES6+
Base de données	MySQL (MariaDB) — base neige_soleil — 12 tables
Accès BDD	PDO avec requêtes préparées — classe Modele
Icônes	Font Awesome 6.0 (CDN)
Serveur local	XAMPP / Apache + phpMyAdmin
Versionnage	Git / GitLab (server-rtit-consulting.com)
Environnement	Windows, VS Code, navigateur Chrome

STRUCTURE DES FICHIERS — ARCHITECTURE MVC

- index.php — Point d'entrée unique (Front Controller) : routage par switch(\$_GET['page']), inclusion dynamique des vues
- Controleur/controleur.class.php — Contrôleur MVC : délégation au modèle, expose les méthodes aux vues
- Modele/modele.php — Modèle MVC : connexion PDO, toutes les requêtes SQL (CRUD appartements, réservations, factures, utilisateurs)
- Vue/vue_accueil.php — Page d'accueil avec hero section, bandeau de features et appartements vedettes
- Vue/vue_appartements.php — Catalogue complet des logements avec cartes et prix à la nuit calculés
- Vue/vue_detail.php — Fiche détaillée d'un appartement avec formulaire de réservation (dates + personnes)
- Vue/vue_connexion.php — Formulaire de connexion email/mot de passe
- Vue/vue_inscription.php — Inscription avec choix du rôle (client ou propriétaire), déclenche les triggers SQL
- Vue/vue_panier.php — Panier de réservations en statut 'panier', calcul du total, suppression, validation
- Vue/vue_profil.php — Espace personnel avec informations utilisateur et historique des réservations validées
- Vue/vue_facture.php — Facture imprimable avec toutes les informations de la réservation
- Vue/vue_mes_apparts.php — Espace propriétaire : liste des appartements avec actions CRUD
- Vue/vue_ajout_appart.php — Formulaire d'ajout d'un nouvel appartement (propriétaire)
- Vue/vue_modif_appart.php — Formulaire de modification d'un appartement existant
- Vue/vue_materiel.php — Catalogue du matériel de sport disponible à la location
- Vue/vue_header.php — En-tête de navigation commune (logo, liens, menu utilisateur)
- Style/ — Feuilles CSS dédiées par page (style_accueil.css, style_appartement.css, style_panier.css...)
- images/ — Photographies des chalets et arrière-plan montagne

BASE DE DONNÉES — STRUCTURE DES TABLES PRINCIPALES

User	Table mère — tous les utilisateurs (id_perso, nom, prenom, email, mdp, role, tel)
CLIENT	Table fille — vacanciers (hérite de User via trigger insert_client)
PROPRIETAIRE	Table fille — propriétaires (hérite de User via trigger insert_proprio, + iban, adresse)
APPARTEMENT	Biens à louer (num_appart, type, surface, capacite, exposition, distance_pistes, prix_hebdo, image, id_proprio)
RESERVATION	Réservations (date_debut_loc, date_fin_loc, nb_personnes, statut : 'panier' ou 'validee')
FACTURE	Factures générées à la validation (num_facture auto, montant_ttc, date_emission, id_reser)
CONTRAT	Mandats de gestion entre l'agence et les propriétaires (1 contrat = 1 appartement)
MATERIEL	Équipements de sport disponibles à la location (ski, rando, prix_jour, état)
LOUER_MATERIEL	Liaisons client ↔ matériel loué avec dates
DISPOSER	Liaisons appartement ↔ équipements disponibles (table de jonction)

4. PAGES ET FONCTIONNALITÉS DE L'APPLICATION

PAGE 1 — ACCUEIL (VUE_ACCUEIL.PHP)

Page d'accueil avec une hero section plein écran affichant l'image des montagnes. Un bandeau de features présente les atouts de l'agence (accès pistes, location matériel, support 7j/7, réservation sécurisée). La section "Nos coups de cœur" affiche les 3 premiers appartements en vedette récupérés dynamiquement depuis la base de données via `getAppartementsVedettes()`. Chaque carte affiche l'image, le type, la surface, la capacité et le prix à la nuit.

PAGE 2 — CONNEXION / INSCRIPTION (VUE_CONNEXION.PHP / VUE_INSCRIPTION.PHP)

La page de connexion authentifie l'utilisateur via email et mot de passe, interroge la table User via PDO et initialise la session PHP (`$_SESSION['id_user']` et `$_SESSION['role']`). L'inscription propose deux parcours : client ou propriétaire. L'insertion dans CLIENT ou PROPRIETAIRE déclenche automatiquement les triggers SQL qui propagent les données dans la table User et attribuent le rôle. Le mot de passe est ensuite mis à jour dans User via une requête UPDATE séparée.

- Authentification via table User unifiée — tous rôles confondus
- Inscription client : trigger `insert_client` → insertion automatique dans User avec `role='client'`
- Inscription propriétaire : trigger `insert_proprio` → insertion dans User avec `role='proprietaire'`
- Session PHP initialisée avec `id_user` et `role` après connexion réussie

PAGE 3 — CATALOGUE (VUE_APPARTEMENTS.PHP)

Affiche l'ensemble des appartements disponibles sous forme de grille de cartes. Chaque carte affiche la photo du chalet, le badge de prix à la nuit (calculé depuis `prix_hebdo / 7`), le type, le numéro, la surface, la capacité d'accueil et un bouton de détail. Le prix par nuit est calculé côté PHP directement dans la vue avec `number_format()`.

PAGE 4 — DÉTAIL ET RÉSERVATION (VUE_DETAIL.PHP)

Fiche complète d'un appartement affichant toutes ses caractéristiques (exposition, distance pistes, surface, capacité). Un formulaire de réservation permet de saisir les dates d'arrivée et de départ ainsi que le nombre de personnes. La soumission envoie vers le cas 'ajout_panier' de `index.php`, qui vérifie via SQL les chevauchements avec les réservations validées existantes avant d'insérer la réservation avec le statut 'panier'.

- Vérification des chevauchements : requête SQL avec `NOT (date_fin <= ? OR date_debut >= ?)`
- Insertion en statut 'panier' pour permettre la modification avant paiement
- Gestion des erreurs de dates (date fin avant date début) côté modèle PHP

PAGE 5 — PANIER (VUE_PANIER.PHP)

Liste toutes les réservations en statut 'panier' du client connecté via une jointure SQL RESERVATION × APPARTEMENT. Le calcul du nombre de jours et du prix par ligne est effectué en PHP (`DateTime::diff()`). Un total général est affiché. Le client peut supprimer une

réservation (suppression physique en base) ou valider le panier, ce qui déclenche la création des factures et le passage au statut 'validee'.

```
// Calcul du prix par ligne dans vue_panier.php
$date1 = new DateTime($uneResa['date_debut_loc']);
$date2 = new DateTime($uneResa['date_fin_loc']);
$diff = $date1->diff($date2);
$nbJours = ($diff->days == 0) ? 1 : $diff->days; $prixLigne = ($uneResa['prix_hebdo'] / 7) * $nbJours;
```

PAGE 6 — PROFIL ET HISTORIQUE (VUE_PROFIL.PHP)

Espace personnel affichant les informations du client (avatar généré depuis les initiales, nom, email, téléphone) et un tableau de l'historique complet des réservations validées avec leur facture. Une requête SQL complexe joint RESERVATION, APPARTEMENT et FACTURE pour afficher le type de bien, les dates, le montant TTC et le numéro de facture. Chaque ligne permet d'accéder à la facture détaillée.

PAGE 7 — FACTURE IMPRIMABLE (VUE_FACTURE.PHP)

Génère une facture complète au format document avec en-tête de l'agence, informations du client, détail du séjour (dates, surface, capacité), calcul du montant HT / TVA 20% / TTC et pied de page avec conditions. Un bouton déclenche `window.print()` pour l'impression ou l'export PDF via le navigateur. La facture est identifiée par un numéro auto-généré (FAC-ANNÉE-id_reser).

PAGES 8 & 9 — ESPACE PROPRIÉTAIRE (VUE_MES_APPARTS / VUE_AJOUT_APPART / VUE_MODIF_APPART)

Interface dédiée aux propriétaires pour gérer leur portefeuille d'appartements. La page `mes_apparts` liste leurs biens avec des boutons Modifier et Supprimer. L'ajout et la modification passent par des formulaires POST traités dans `index.php` qui appellent les méthodes du contrôleur (`ajouterAppartement`, `modifierAppartement`, `supprimerAppartement`). La suppression inclut une vérification que l'appartement appartient bien au propriétaire connecté (double condition `WHERE id_appart = ? AND id_propio = ?`).

5. CIRCONSTANCES DE RÉALISATION

Contexte	Formation BTS SIO SLAM — Projet "Client Léger" Neige & Soleil
Type de mission	Projet individuel de développement full-stack en architecture MVC
Période	2025 — 2026
Équipe	Projet individuel
Environnement	Windows, VS Code, XAMPP / phpMyAdmin, navigateur Chrome
Vernionnage	Git / GitLab (server-rtit-consulting.com)
Méthode	Développement fonctionnalité par fonctionnalité, tests manuels

DÉROULEMENT DU PROJET

- Phase 1 — Analyse du besoin, conception du schéma de base de données (12 tables, héritage par triggers)
- Phase 2 — Mise en place de l'architecture MVC : index.php (Front Controller), classe Controleur, classe Modele
- Phase 3 — Développement du module authentification : connexion, inscription client/propriétaire, triggers SQL
- Phase 4 — Développement du catalogue et de la page détail avec formulaire de réservation
- Phase 5 — Développement du panier : ajout, suppression, calcul des prix, vérification des chevauchements
- Phase 6 — Développement de la validation et de la facturation automatique (génération num_facture, statut 'validee')
- Phase 7 — Développement de l'espace propriétaire (CRUD complet sur les appartements)
- Phase 8 — Développement de la facture imprimable avec calcul HT/TVA/TTC et CSS d'impression
- Phase 9 — Tests complets, corrections de bugs, responsive CSS, documentation technique

DIFFICULTÉS RENCONTRÉES ET SOLUTIONS APPORTÉES

Héritage de tables SQL	Mise en place de triggers BEFORE INSERT/UPDATE/DELETE sur CLIENT et PROPRIETAIRE pour synchroniser automatiquement la table mère User
Chevauchement de réservations	Requête SQL avec condition NOT (date_fin <= ? OR date_debut >= ?) sur les réservations au statut 'validee' uniquement
Calcul des prix	Calcul côté PHP avec la classe DateTime et diff() pour obtenir le nombre de jours exact et éviter les erreurs de dates identiques
Sécurité CRUD propriétaire	Double condition WHERE id_appart = ? AND id_proprio = ? sur toutes les opérations de modification et suppression

Doublon de factures	Vérification préalable <code>SELECT COUNT(*)</code> avant chaque <code>INSERT</code> dans <code>FACTURE</code> pour éviter les factures en double lors de rechargements
Numéro de facture unique	Génération automatique du format <code>FAC-ANNÉE-id_reser</code> garantissant l'unicité et la traçabilité de chaque facture
Statut de réservation	Distinction claire entre statut 'panier' (modifiable, supprimable) et 'validee' (immuable, facturée) pour la cohérence des données

LIVRABLE FINAL

L'application Neige & Soleil — Client Léger a été livrée sous forme d'archive ZIP contenant le code source PHP/CSS organisé en architecture MVC, le script SQL complet (`bdd.sql`) avec création des tables, triggers et données de démonstration (10 appartements), ainsi que les assets images. Le projet est versionné sur GitLab.

6. BILAN ET COMPÉTENCES ACQUISES

RÉSULTATS OBTENUS

L'application Neige & Soleil est une plateforme web complète et fonctionnelle qui couvre l'intégralité du cycle de réservation : consultation du catalogue, sélection et réservation d'un logement, gestion du panier, validation avec facturation automatique, et consultation de l'historique. L'espace propriétaire offre un CRUD complet sur les biens. L'architecture MVC garantit une séparation claire des responsabilités et la maintenabilité du code.

COMPÉTENCES TECHNIQUES ACQUISES

PHP / Architecture MVC	Implémentation d'un Front Controller, séparation Modèle/View/Contrôleur, classe PHP avec attributs privés
MySQL / Triggers SQL	Architecture d'héritage de tables avec triggers BEFORE INSERT/UPDATE/DELETE, requêtes complexes avec JOIN multiples
PDO / Requêtes préparées	Connexion sécurisée, requêtes paramétrées, protection contre les injections SQL, fetchAll/fetch avec FETCH_ASSOC
Logique métier PHP	Calcul de durées avec DateTime, gestion des statuts de réservation, génération de numéros de facture
HTML5 / CSS3 responsive	Architecture multi-fichiers CSS, variables CSS, Flexbox, CSS d'impression (@media print) pour la facture
Sécurité applicative	Protection des routes par vérification de session, double condition sur les opérations CRUD propriétaire
Git / GitLab	Versionnage du projet, commits réguliers, dépôt hébergé sur serveur GitLab dédié

COMPÉTENCES TRANSVERSALES

- Conception d'une architecture applicative multi-couches (MVC) sans framework
- Modélisation d'une base de données relationnelle avec 12 tables et relations complexes
- Implémentation de triggers SQL pour gérer l'héritage de tables
- Gestion complète du cycle de vie d'une réservation (panier → validation → facturation)
- Développement d'une interface utilisateur responsive adaptée à plusieurs profils
- Tests et débogage d'une application multi-rôles avec données réelles

LIEN AVEC LE RÉFÉRENTIEL BTS SIO SLAM

B1 — Gérer le patrimoine informatique	Architecture MVC, script SQL structuré, versionnage Git, séparation des fichiers par rôle
B2 — Répondre aux incidents	Gestion des erreurs PDO, vérification des dates, contrôle des chevauchements, doublon de factures
B3 — Développer la présence en ligne	Application web multi-pages responsive, interface adaptée à plusieurs rôles utilisateurs
B4 — Travailler en mode projet	Développement itératif par fonctionnalité, organisation MVC, livrables successifs

B6 — Développement professionnel

Maîtrise de l'architecture MVC sans framework, triggers SQL avancés, logique de facturation

RESSOURCES ET LIENS

GitLab	server-rtit-consulting.com/b1-spring-2025/sujet02_legalhelp
Base de données	MySQL (MariaDB) — neige_soleil — 12 tables + triggers
Technologies	PHP 8.x · MySQL · PDO · HTML5 · CSS3 · Font Awesome 6.0
Contexte	BTS SIO SLAM — Projet Client Léger — 2025/2026

Ce projet démontre la maîtrise de l'architecture MVC en PHP sans framework, la conception d'une base de données relationnelle avec héritage de tables géré par triggers SQL, et le développement d'une logique métier complète couvrant le cycle de réservation de bout en bout — de la consultation du catalogue jusqu'à la génération de la facture imprimable.